



Software Development

Methodologies

Introduction



- Software Engineering is the systematic application of engineering principles to the design, development, testing, deployment, and maintenance of software systems.
- In simple terms:
 - It's about building high-quality, reliable, and maintainable software in a structured, disciplined way — not just coding!
- **Goals of Software Engineering**
 - Build software that works correctly and efficiently
 - Ensure it's maintainable, scalable, and secure
 - Deliver software on time and within budget
 - Satisfy user needs and business requirements.

Software

- program → source code / executable
- dependencies → libraries / frameworks / resources
- documentation
- also known as package
- eg. android → .apk , iphone → .ipa

Building

- compiling code
- package all dependencies
- eg. ant, maven, gradle

Principles of Software Engineering



- Modularity: Divide system into independent modules.
- Abstraction: Focus on what a system does, not how.
- Encapsulation: Keep data and functions together.
- Scalability: Software should handle growth. → vertical / horizontal
- Reusability: Reuse existing code to save effort.
- Maintainability: Code should be easy to modify and fix.

} class

Characteristics of good software



Operational

- Budget: Software should be developed within the budget
- Usability: Software should be usable by the end user
- Efficiently: Software should efficiently use the storage and space
- Correctness: Software should provide all the functionalities correctly without having any bug/issue
- Functionality: Software should meet all the requirements of the user
- Dependability: Software should contain all the dependencies in its package
- Security: Software should keep the data safe from any external threat → CIA triad
- Safety: Software should not be hazardous or harmful to the environment

Transitional

- Portability: If the software can perform the same operations on different environments and platforms, that shows its Portability
- Interoperability: It is the ability of the software to use the information transparently
- Reusability: If on doing slight modifications to the code of the software, we can use it for a different purpose, then it is reusable
- Adaptability: It is an ability to adapt the new changes in the environment

Maintenance

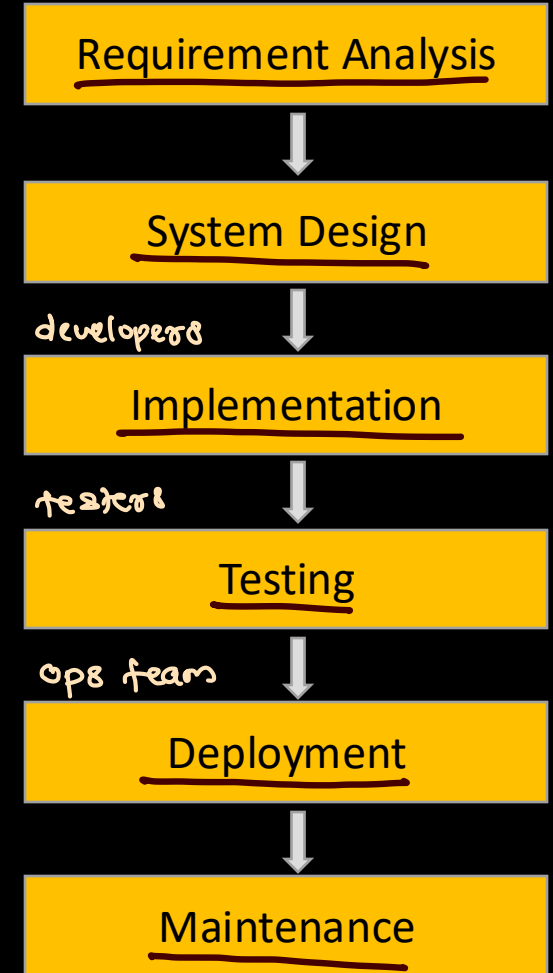
- Maintainability: The software should be easy to maintain by any user
- Flexibility: The software should be flexible to any changes made to it
- Extensibility: There should not be any problem with the software on increasing the number of functions performed by it
- Testability: It should be easy to test the software
- Modularity: A software is of high modularity if it can be divided into separate independent parts and can be modified and tested separately



Software Development **Life Cycle**

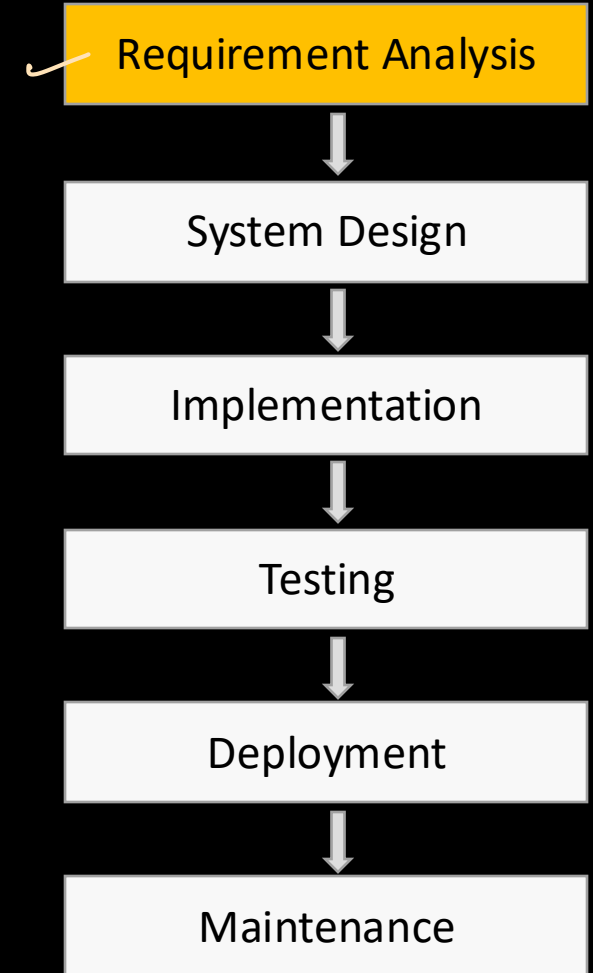
Overview

- Also called as Software Development Process
- Is a well-defined, structured sequence of stages in software engineering to develop the intended software product
- Is a framework defining tasks performed at each step in the software development process
- Aims to produce a high-quality software that
 - meets or exceeds customer expectations
 - reaches completion within times and cost estimates
- Consists of detailed plan (stages) of describing how to develop, test, deploy and maintain the software product



Requirement Analysis (Requirement Gathering)

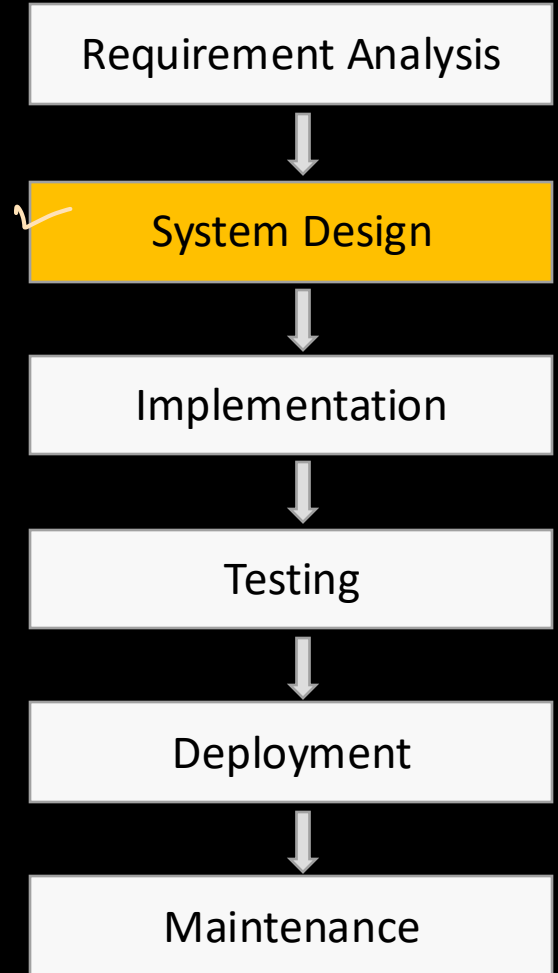
- Goal: Understand what needs to be built
- Activities:
 - Gather requirements from stakeholders, clients, or end users
 - Create a Software Requirement Specification (SRS) document
 - Identify technical and business feasibility
- Output: SRS document



System Design

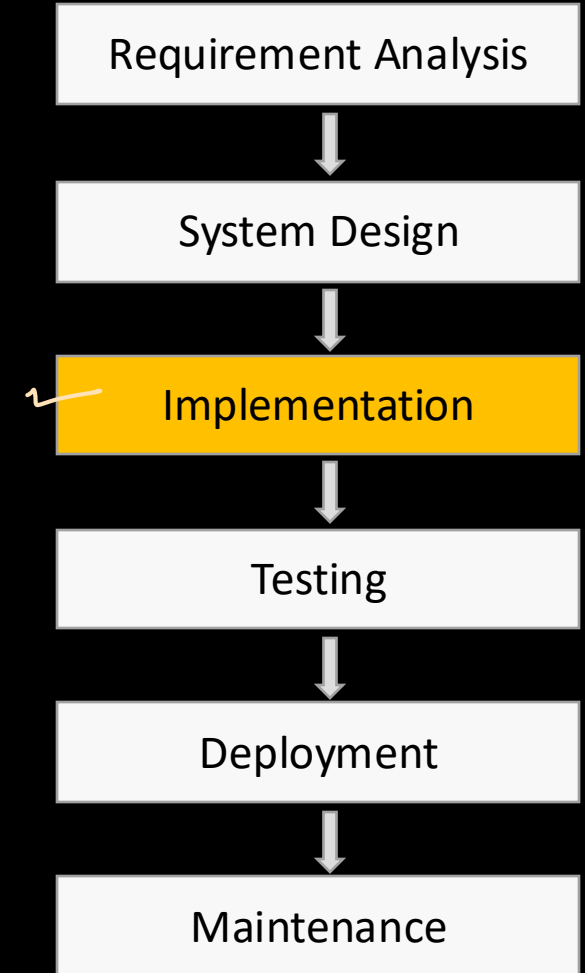
- **Goal:** Plan how the software will be built
- **Activities:**
 - Define architecture, data flow, and database schema
 - Create UI/UX wireframes
 - Select technologies (frontend, backend, database, etc.)
- **Output:** Design Documents (HLD, LLD — High/Low-Level Design)

↓
DDS → Design Document Specification



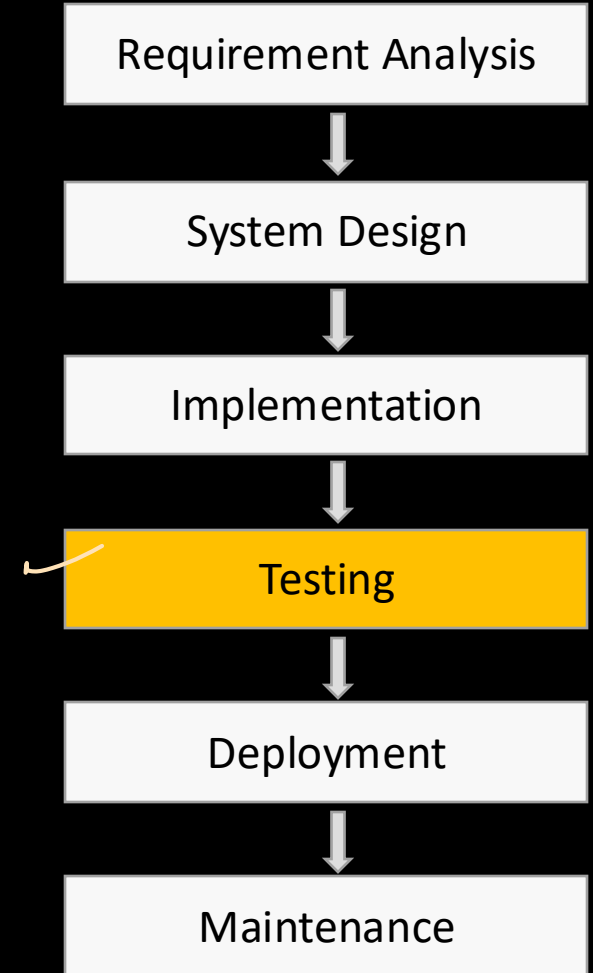
Implementation → coding / programming / Building

- Goal: Convert design into working code
- Activities:
 - Developers write code following coding standards
 - Use version control (Git) for collaboration
 - Conduct unit testing for each module
- Output: Working source code



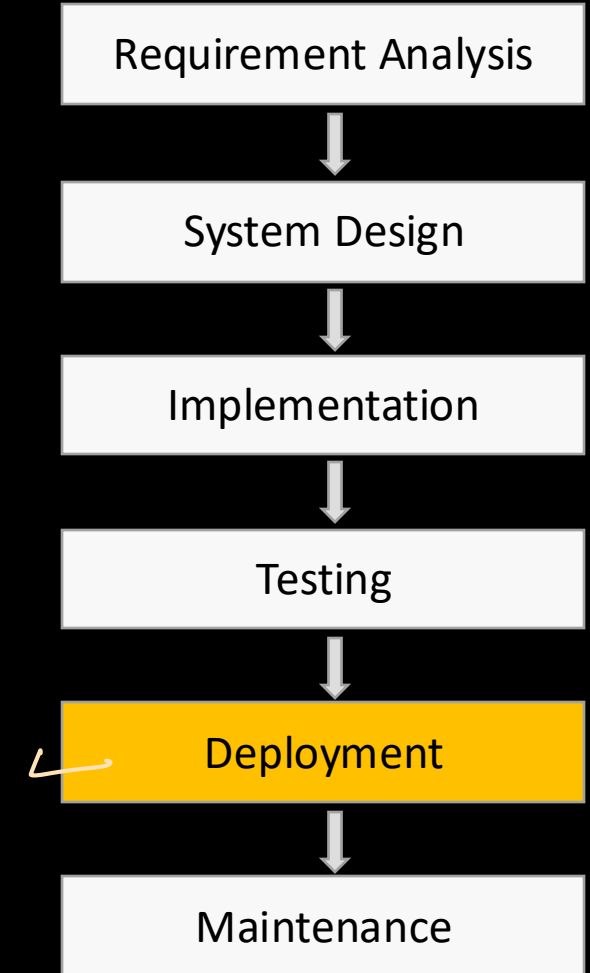
Testing

- **Goal:** Ensure software works correctly and meets requirements
- **Activities:**
 - Perform unit, integration, system, and user acceptance testing (UAT)
 - Identify and fix bugs
 - Validate performance, security, and compatibility
- **Output:** Tested and verified application



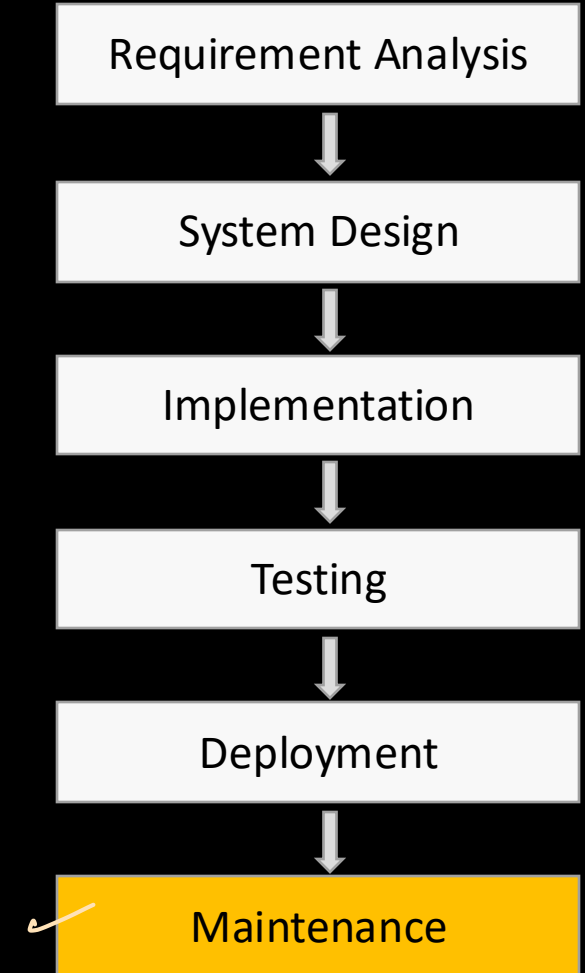
Deployment

- **Goal:** Deliver the product to the end users or production environment
- **Activities:**
 - Deploy software using CI/CD pipelines or manual methods
 - Conduct smoke testing in production
 - Monitor for early issues
- **Output:** Software available to users



Maintenance

- **Goal:** Keep the software running smoothly after release
- **Activities:**
 - Fix bugs reported by users
 - Update software with new features or security patches
 - Improve performance
- **Output:** Updated and stable software



SDLC Models



Model	Description	When to Use
<u>Waterfall</u>	Linear and sequential — <u>each phase follows the previous one.</u>	<u>Small, well-defined projects.</u>
<u>Agile</u>	Iterative and incremental — work in short sprints with continuous feedback.	Dynamic projects with evolving requirements.
<u>Spiral</u>	Combines design and prototyping with risk analysis.	Large, complex, and high-risk projects.
<u>V-Model</u>	Extension of Waterfall with testing after every phase.	When strict validation is required.
<u>Iterative</u>	Develop a version, test, improve, and repeat.	When gradual improvement is needed.
<u>DevOps</u>	Continuous integration, delivery, and monitoring.	For modern, cloud-based environments needing fast releases.



Benefits of SDLC

- Ensures **structured development** and **predictability**
- Improves **project planning** and **resource management**
- Reduces **project risks** and **development costs**
- Delivers **high-quality** software that meets requirements



Agile Methodologies

Agile Development

→ step by step



- Agile Development is a flexible, iterative approach to software development that focuses on continuous delivery, collaboration, and customer feedback
- Instead of building the entire product in one go (like in the Waterfall model), Agile breaks the project into small parts called iterations or sprints – each delivering a working piece of software
- Agile = “Build small, deliver fast, get feedback, and improve continuously.”
- The Agile Manifesto (Core Philosophy)
 - In 2001, 17 software developers wrote the Agile Manifesto, which defines 4 key values and 12 principles for better software development

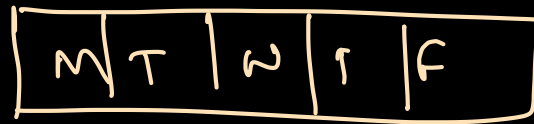
Traditional Focus	Agile Focus
<u>Processes and tools</u>	<u>Individuals and interactions</u>
<u>Comprehensive documentation</u>	<u>Working software</u>
<u>Contract negotiation</u>	<u>Customer collaboration</u>
<u>Following a plan</u>	<u>Responding to change</u>

sprint → time boxed event
→ one to four weeks of time

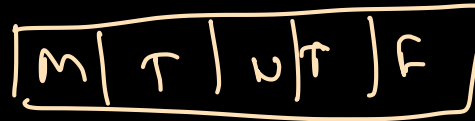
Components of Agile Development



Component	Description
✓ <u>Sprint</u>	<u>A short, time-boxed period (1–4 weeks) where a small part of the product is developed.</u>
✓ <u>Product Backlog</u>	<u>A prioritized list of features or user stories that need to be implemented.</u>
✓ <u>Sprint Backlog</u>	<u>Subset of the product backlog selected for a specific sprint.</u>
✓ <u>Daily Stand-up</u>	A short 15-minute daily meeting to track progress and obstacles.
✓ <u>Sprint Review</u>	Meeting at the end of the sprint to demonstrate completed work.
✓ <u>Sprint Retrospective</u>	A reflection session to improve team processes in the next sprint.

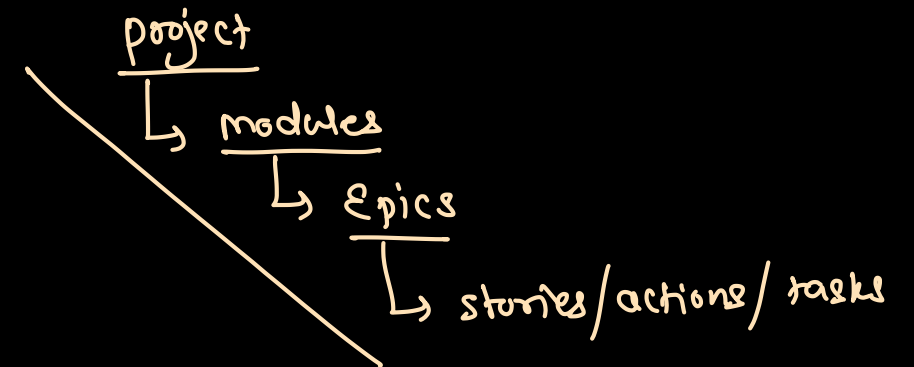


Sprint 1



Sprint 2

.....



product backlog → collection of all the stories

* e-commerce

- user module → * Registration of user ← Epic

- product module

- order module

- API module

- Notification module

① → create a table named users

② → create an API - /users/register

③ → create frontend page - /register

④ → connect component to API

← story → action/task

* Authenticate user ← Epic

⑤ → create an API - /users/login

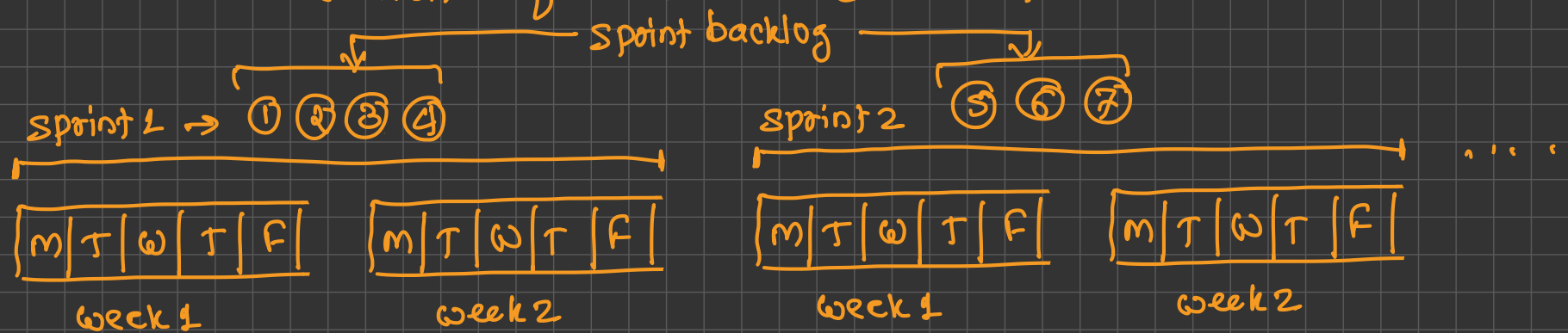
⑥ → create frontend page - /login

⑦ → connect component to API

* forgot password ← Epic

...

collection of stories selected in a sprint



sprint → 2 weeks

sprint → time boxed event (1-4 weeks)

sprint → 2 weeks

Agile Frameworks



Framework	Highlights
Scrum	Most popular Agile framework; uses sprints, roles, and ceremonies.
Kanban	Visual workflow (board with “To Do → Doing → Done”) focusing on continuous flow.
Extreme Programming (XP)	Emphasizes coding practices like pair programming and continuous integration.
Lean	Focuses on minimizing waste and maximizing value delivery.
SAFe (Scaled Agile Framework)	Extends Agile for large, enterprise-level teams.

What is Scrum ?



- Scrum is an Agile framework used to develop, deliver, and sustain complex software projects
- It emphasizes **team collaboration, iterative progress, continuous feedback, and flexibility**
- Scrum helps teams deliver **working software quickly, in small increments**, while continuously improving
- Scrum divides the entire project into **small, manageable chunks** called **Sprints** (typically 2–4 weeks)
- Each sprint results in a **potentially shippable product increment** — something working and usable
- At the end of every sprint:
 - The team reviews progress
 - The product is demonstrated
 - Feedback is gathered
 - Improvements are planned for the next sprint

Scrum Roles



- **Product Owner**
 - Represents the customer or stakeholders
 - Manages the Product Backlog
 - Prioritizes features and defines User Stories
 - Ensures the team builds what provides the most value
- **Scrum Master**
 - Acts as a coach and facilitator for the Scrum team
 - Removes obstacles (impediments)
 - Ensures Scrum principles and ceremonies are followed
 - Shields team from distractions
- **Development Team**
 - Cross-functional members (developers, testers, designers, etc.)
 - Self-organizing — decide how to build the product
 - Delivers a working product increment every sprint



Scrum Artifacts (Key Documents)

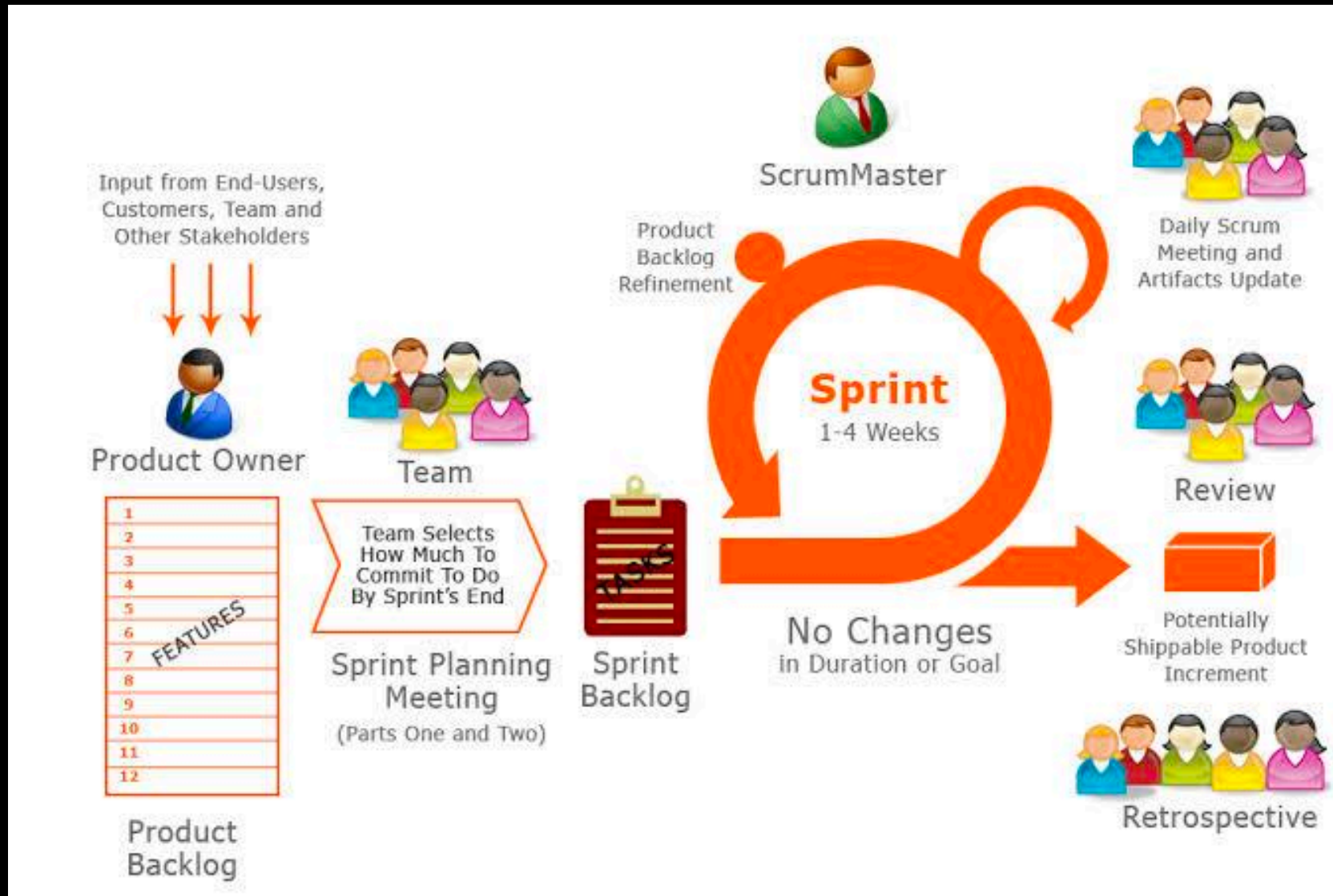
- **Product Backlog**
 - A prioritized list of everything needed in the product (features, bugs, improvements)
 - Managed by the Product Owner
- **Sprint Backlog**
 - A subset of the Product Backlog selected for the current sprint
 - Includes tasks and a goal
- **Increment**
 - The working product that results from a sprint — it must be usable and meet the “Definition of Done”
- **Definition of Done (DoD)**
 - A shared understanding of what “done” means (e.g., coded, tested, documented, deployed)

Scrum Ceremonies (Events)



Ceremony	Purpose	Duration
Sprint Planning	Define the Sprint Goal and decide which backlog items to work on.	2–4 hours (for 2-week sprint)
Daily Scrum (Stand-up)	15-min meeting where team members share: 1. What they did yesterday 2. What they'll do today 3. Any blockers	15 minutes daily
Sprint Review	Demonstrate the completed work to stakeholders and collect feedback.	1–2 hours
Sprint Retrospective	Discuss what went well , what didn't , and how to improve next sprint.	1–1.5 hours
The Sprint	The actual 2–4 week development cycle where work happens.	Fixed timebox

Scrum Workflow



Scrum tools



- **Jira** – <https://www.atlassian.com/software/jira/>
- **Clarizen** – <https://www.clarizen.com/>
- **GitScrum** – <https://site.gitscrum.com/>
- **Vivify Scrum** – <https://www.vivifyscrum.com/>
- **Yodiz** – <https://www.yodiz.com/>
- **ScrumDo** – <https://www.scrumdo.com/>
- **Quickscrum** – <https://www.quickscrum.com/>
- **Manuscript** – <https://www.manuscript.com/>
- **Scrumwise** – <https://www.scrumwise.com/>
- **Axosoft** – <https://www.axosoft.com/>



Application Testing

Introduction



- **Application Testing** is the **process of verifying and validating** that a software application:
 - Works as intended
 - Meets business and user requirements
 - Is free from defects or bugs
 - Performs well under various conditions
- In simple terms:
 - Application Testing ensures the **software is reliable, secure, and user-friendly** before it goes live
- **Goals of Application Testing**
 - Detect and fix **bugs** early
 - Ensure the **functionality** meets requirements
 - Verify **performance, usability, and security**
 - Prevent future defects through **regression tests**
 - Ensure **cross-platform** and **cross-browser compatibility** (for web & mobile apps)

Phases of Application Testing



Phase	Description
Test Planning	Define test objectives, strategy, tools, scope, and resources.
Test Design	Create test cases , test data , and define expected outcomes.
Test Environment Setup	Prepare hardware, software, and network needed for testing.
Test Execution	Run test cases and record results.
Defect Reporting & Tracking	Log bugs in tools (like Jira, Bugzilla).
Retesting & Regression Testing	Re-test after bug fixes and ensure no new issues appear.
Test Closure	Prepare summary report and finalize testing.

Application Testing in SDLC



SDLC Phase	Testing Involvement
Requirement	Review for testability.
Design	Verify architecture and data flow.
Implementation	Unit & integration testing.
Testing	Functional, non-functional testing.
Deployment	Smoke testing in production.
Maintenance	Regression & patch testing.



Types of Application Testing

- **Manual Testing**

- Testers execute test cases **without automation tools**
- Focus on **exploratory, usability, and ad-hoc** testing
- Suitable for **UI validation** and **small projects**
- Example tools: Excel sheets, TestRail for documentation

- **Automated Testing**

- Test cases are executed using **automation tools or scripts**
- Great for **repetitive, regression, and performance** tests
- Saves time and improves accuracy
- Example tools: Selenium, Cypress, Appium, JUnit, PyTest

Levels of Application Testing



Level	Purpose	Performed By	Example Tools
Unit Testing	Test individual modules or functions.	Developers	JUnit, NUnit, PyTest
Integration Testing	Verify how modules interact together.	Developers/Testers	Postman, JUnit
System Testing	Test the complete application as a whole.	QA Team	Selenium, TestNG
User Acceptance Testing (UAT)	Validate the system with real-world scenarios.	End Users / Clients	Manual / UAT tools

Types of Testing: Functional



Type	Description
Smoke Testing	Quick test to ensure basic functions work (build verification).
Sanity Testing	Check new functionality or bug fixes.
Regression Testing	Ensure new changes don't break existing features.
Interface Testing	Validate communication between systems or APIs.
System Testing	End-to-end testing of the entire system.
User Acceptance Testing (UAT)	Done by end-users to validate business requirements.

Types of Testing: Non-Functional



Type	Description
Performance Testing	Measure speed, scalability, and stability.
Load Testing	Check system performance under expected load.
Stress Testing	Push beyond limits to see where it breaks.
Security Testing	Ensure data protection and access control.
Usability Testing	Verify ease of use and user experience.
Compatibility Testing	Ensure app works on different OS, browsers, and devices.
Accessibility Testing	Ensure app is usable by people with disabilities.

Testing Methods



Testing Method	Tester's Knowledge	Focus Area	Performed By	SDLC Phase
White Box Testing	Full knowledge of internal code, logic, and structure	Internal workings (logic, code, paths)	Developers	Implementation/ Unit testing
Black Box Testing	No knowledge of internal code	Functionality and output	Testers / QA	Testing/ UAT
Gray Box Testing	Partial knowledge of internal structure	Combination of internal and external	Developers & Testers	Testing/Integration

Common Tools for Application Testing



Category	Popular Tools
Functional Testing	Selenium, Cypress, Playwright, TestNG
API Testing	Postman, SoapUI, Rest Assured
Performance Testing	JMeter, Gatling, LoadRunner
Mobile Testing	Appium, Espresso, XCUITest
Security Testing	OWASP ZAP, Burp Suite
Test Management	Jira, TestRail, Zephyr
CI/CD Integration	Jenkins, GitHub Actions



Benefits of Application Testing

- Ensures software quality and reliability
- Detects defects early, reducing costs
- Enhances user satisfaction
- Improves security and performance
- Supports continuous delivery (CI/CD) pipelines



Challenges in Application Testing

- Handling frequent code changes in Agile
- Testing across many browsers/devices
- Managing complex test data
- Balancing speed vs. depth in automated tests